Week 05 — Chapter III: Memory Management

> Slides: week05-chapter-3-memory-management.pdf Assignment: task-paging-simulation-using-python.pdf

III.1. Overview

Memory management is an important responsibility of the OS. It ensures that processes **have access to memory when needed**, **prevents interference** between processes, and **optimizes system performance**.

Outline

1. What is Memory Management?
2. Memory Hierarchy
3. Memory Allocation Techniques
4. Conclusion
5. Lab

III.2. What is Memory Management?

> **Memory management** is the process of **allocating, tracking, and freeing memory** in a computer system to ensure efficient execution of programs.

The four main goals:

Goal	Meaning
Allocation	Provides memory to processes when needed
Protection	Prevents processes from accessing **unauthorized** memory
Sharing	Allows **controlled** memory sharing between processes
Optimization	**Minimize fragmentation** and **maximize utilization**

Protection is what makes the process **isolation** from Chapter II real: one process cannot read another's memory because the OS and hardware don't map it.

III.3. Memory Hierarchy

> The **memory hierarchy** is the structured arrangement of different types of memory based on **speed, cost, and capacity**. It optimizes performance by keeping the **most frequently accessed data in the fastest memory available**.

It is characterized by:

- **Response time:** faster memory is placed **closer to the CPU** to reduce access time.
- **Cost vs capacity:** faster memory (like cache) is **more expensive and smaller**; slower memory (like hard drives) is **cheaper and larger**.
- **Locality of reference:** programs tend to access the same data repeatedly (**temporal locality**) or data near recently accessed data (**spatial locality**), which **justifies** storing it in faster memory.

Levels of the hierarchy

Level	Speed	Size	Cost per bit	Example
CPU registers	Fastest	Very small	Very high	Temporary computation storage
Cache (L1, L2, L3)	Very fast	Small	High	Frequently used instructions
Main memory (RAM)	Moderate	Medium	Moderate	Active programs and data
Secondary storage	Slow	Large	Low	HDDs, SSDs
Tertiary storage	Slowest	Very large	Very low	Backup tapes, optical disks
Virtual memory	Slowest	Very large	Lowest	Swap space on disk

> ⚠️ The columns in the slide's version of this table are **shifted out of line** (speeds and examples don't sit on the right rows). The table above is the corrected mapping, which follows from "faster = smaller and more expensive".

The pattern to remember: as you go down, **speed falls, size grows, cost per bit falls**. Locality of reference is what makes the whole arrangement work.

III.4. Memory Allocation Techniques

1. Contiguous memory allocation

Processes are allocated memory in **continuous blocks**.

A. Fixed partitioning — memory is divided into **fixed-size partitions at boot time**.

Characteristics	Problems
Each partition holds **exactly one process**	**Internal fragmentation:** unused memory **inside** a partition when the process is smaller than it
Partition sizes may be equal or unequal	**Limitation:** large processes may not fit if partitions are too small

B. Dynamic partitioning — memory is allocated **as needed in variable-sized blocks**.

Characteristics	Problems
Each process gets **exactly the memory it requires**	**External fragmentation:** small gaps **between** allocated blocks that cannot be used
More flexible than fixed partitioning	**Compaction** is required periodically to consolidate free space

> **Internal vs external fragmentation** — the classic exam pair: **Internal** = wasted space **inside** an allocated block (fixed partitioning, and the last page in paging). **External** = wasted space **between** allocated blocks (dynamic partitioning, segmentation).

2. Non-contiguous memory allocation

Processes are allocated memory in **non-adjacent blocks**.

Type	Characteristics	Advantages	Disadvantages
Paging	Divides memory into **fixed-size blocks called frames**, and processes into **same-sized blocks (pages)**	**No external fragmentation**; allows **virtual memory** (pages can be swapped to disk)	Small **internal** fragmentation possible (the last page may not be full); overhead of **page table** management
Segmentation	Divides memory into **variable-sized segments matching program structure** (code, data, stack); uses a **segment table** with base addresses and lengths	**Matches the programmer's view** of memory; **easy sharing** of segments (e.g. shared libraries)	**External fragmentation** can occur; complex memory management
Segmented paging (hybrid)	Combines segmentation and paging: **segments are divided into pages**	Benefits of both	Used in modern systems such as **x86**

Pages vs frames: a **page** is a fixed-size block of a **process's logical memory**; a **frame** is the same-size block of **physical memory**. The **page table** maps page → frame, which is what makes non-contiguous allocation invisible to the program.

3. Virtual memory techniques

> **Virtual memory** is a mechanism using **hardware and software** where **part of secondary storage** gives the user the **illusion that it is part of main memory**. It helps run multiple applications with **low main memory** and increases the **degree of multiprogramming**. It is commonly implemented using **demand paging**.

Technique	Characteristics
Demand paging	Pages are loaded into memory **only when needed**. Uses **page faults** to trigger loading from disk
Page replacement algorithms	Used when **no free frames** are available

Page replacement algorithms:

Algorithm	Rule	Note
FIFO (First-In-First-Out)	Replaces the **oldest** page	Simple but **poor performance**
LRU (Least Recently Used)	Replaces the **least recently accessed** page	**Near-optimal** but **expensive to implement**
Optimal	Replaces the page **not needed for the longest time**	Needs the future, so it's a **theoretical benchmark**

A **page fault** is not an error: it's the hardware telling the OS "this page isn't in memory yet", and the OS loads it.

4. Buddy system allocation

Divides memory into **power-of-2 sized blocks**.

Process:

1. On a request, the system finds the **smallest sufficient block**.
2. If none is available, it **splits a larger block into two "buddies"**.
3. When memory is freed, it **merges adjacent free buddies**.

Used in the **Linux kernel** for efficient allocation.

5. Slab allocation

Allocates memory for **kernel objects**. It **pre-allocates "slabs"** of memory for **frequently used objects**.

Advantages: no fragmentation for small objects, and **fast allocation/deallocation**.

Comparison of techniques

Technique	Contiguous?	Fragmentation	Overhead	Used for
Fixed partitioning	Yes	Internal	Low	Early OS
Dynamic partitioning	Yes	External	Medium	—
Paging	Yes / No	Small internal	Medium	Modern OS
Segmentation	Yes / No	External	High	Program memory
Buddy system	Yes / No	Minimal	Medium	Kernel memory
Slab allocation	No	None	Low	Kernel objects

III.5. Conclusion

Memory management is **essential for system stability and performance**. The chapter introduced the **memory hierarchy** and explained **paging, segmentation** and the other techniques used in different cases.

Lab: simulating paging, and memory-leak detection

1. Paging

> Paging divides memory into **fixed-size blocks (pages)**. It allows **non-contiguous** allocation and **avoids external fragmentation**. **Simulation idea:** use a **list** to represent physical memory, and **map logical pages to physical frames**.

The slide's program:

```
import random

# Physical memory: 8 frames (some already occupied)
physical_memory = [None, 'OS', None, 'AppX', None, None, 'OS', None]

# Our program's logical pages
program_pages = ['A', 'B', 'C', 'D']   # Program thinks it has pages 0,1,2,3
page_table = {}                        # logical page -> physical frame
available_frames = [i for i, frame in enumerate(physical_memory) if frame is None]

# Load program pages into available physical frames
for logical_page_num, page_content in enumerate(program_pages):
    if available_frames:
        physical_frame_num = random.choice(available_frames)   # any free frame will do
        available_frames.remove(physical_frame_num)
        page_table[logical_page_num] = physical_frame_num
        physical_memory[physical_frame_num] = page_content

# Simulate a memory access
logical_address = 1
physical_frame = page_table[logical_address]
print(f"Page table says: go to frame {physical_frame}")
print(f"Content found: '{physical_memory[physical_frame]}'")
```

Example output from the slides:

```
Initial Memory: [None, 'OS', None, 'AppX', None, None, 'OS', None]
Available frames: [0, 2, 4, 5, 7]

After loading program:
Page Table (logical -> physical): {0: 5, 1: 0, 2: 2, 3: 4}
Physical Memory: ['B', 'OS', 'C', 'AppX', 'D', 'A', 'OS', None]

Program asks for page 1
Page table says: go to frame 0
Content found: 'B'
```

What the simulation demonstrates:

- The program's pages land in frames **0, 2, 4, 5** — **not adjacent**, and not in order. That is **non-contiguous allocation**, and the program never notices.
- Every access goes **through the page table**: logical page → physical frame. That indirection is the whole idea of paging.
- Frames already holding `'OS'` and `'AppX'` are never touched, which is **protection** in action.

> **Two typos in the slide code to watch for if you copy it:** it declares `spage_table = {}` but then uses `page_table`, and it writes `logical_address = 2 [ change values here]`, which is a note rather than valid Python (the printed output is for page 1).

2. Memory leak detection with `tracemalloc`

> Memory leaks can **degrade performance or crash applications**. `tracemalloc` helps **pinpoint the source** of leaks by tracking allocations over time.

```
import tracemalloc

tracemalloc.start()
snap1 = tracemalloc.take_snapshot()          # before

data = ["leak" * 100 for _ in range(500)]    # suspect code

snap2 = tracemalloc.take_snapshot()          # after
for stat in snap2.compare_to(snap1, 'lineno')[:3]:
    print(f"Potential leak: {stat}")
tracemalloc.stop()
```

The method is **snapshot → run suspect code → snapshot → compare**. `compare_to(..., 'lineno')` sorts the differences by source line, so the top entries point at the lines that allocated the most new memory.

Most important for exams

- **Four goals of memory management: allocation, protection, sharing, optimization**.
- **Memory hierarchy order:** registers → cache (L1/L2/L3) → RAM → secondary (HDD/SSD) → tertiary (tapes/optical) → virtual memory (swap). **Down the list: slower, bigger, cheaper per bit.**
- **Locality of reference: temporal** (same data again) and **spatial** (nearby data) — the justification for caching.
- **Internal fragmentation = waste inside a block** (fixed partitioning, last page). **External fragmentation = waste between blocks** (dynamic partitioning, segmentation). Expect to be asked which technique causes which.
- **Fixed vs dynamic partitioning:** fixed = set at boot, internal fragmentation; dynamic = exact size, external fragmentation, needs **compaction**.

- **Paging:** fixed-size **pages** → **frames**, via a **page table**. **No external fragmentation**, small internal fragmentation, page-table overhead, and it **enables virtual memory**.
- **Segmentation: variable-size** segments matching program structure (code, data, stack), **segment table with base + length**, matches the programmer's view, easy sharing, but **external fragmentation**.
- **Segmented paging:** segments divided into pages; used by **x86**.
- **Virtual memory** = secondary storage used as if it were main memory; raises the **degree of multiprogramming**; implemented with **demand paging** and **page faults**.
- **Page replacement: FIFO** (oldest, simple, poor), **LRU** (least recently used, near-optimal, expensive), **Optimal** (farthest future use, theoretical benchmark).
- **Buddy system:** power-of-2 blocks, **split and merge buddies**, used in the **Linux kernel**. **Slab:** pre-allocated slabs for **kernel objects**, no fragmentation for small objects, fast.
- **Know the comparison table** (contiguous? / fragmentation / overhead / used for) — it's a ready-made exam question.